

DEFINICIÓN DEL PROYECTO — TUHOSPEDAJE

1. Identidad, Nicho y Propósito del Negocio

TuHospedaje es una plataforma web centralizada de reservas de alojamiento enfocada en **alojamientos turísticos**. El propósito principal del negocio es resolver la fricción entre los usuarios que buscan hospedaje de calidad y los administradores que necesitan gestionar su catálogo de manera eficiente. La plataforma permite a los clientes finales explorar un catálogo dinámico y detallado, mientras que provee a los administradores un panel de control intuitivo para crear, actualizar y dar de baja alojamientos.

- **Nombre Oficial:** TuHospedaje
- **Identidad Visual:** Isologotipo institucional con soporte nativo para esquemas de colores claro y oscuro.
- **Ubicación del recurso:** `frontend/src/assets/images/TuHospedaje_Isologotipo.png`

2. Alcance del Proyecto y Hoja de Ruta (Roadmap)

El desarrollo se ejecutará de forma incremental a lo largo de **4 Sprints** planificados, garantizando un Producto Mínimo Viable (MVP) funcional desde el primer hito.

2.1. Matriz de Incremento por Sprint

Sprint	Foco Estratégico	Entregables Principales
Sprint 1	Base de la Solución	Catálogo interactivo de alojamientos, Panel de Administración (CRUD), identidad visual y maquetación responsiva.
Sprint 2	Seguridad y Organización	Autenticación/Registro con JWT, sistema de roles (Admin/User), categorización dinámica y gestión de características.
Sprint 3	Búsqueda y UX Avanzada	Buscador predictivo con filtros funcionales, sección de favoritos, módulo de políticas del alojamiento y sistema de puntuaciones/reseñas.
Sprint 4	Transaccionalidad	Motor de reservas (Booking), control de disponibilidad, historial de usuario, notificaciones por Email y canal de contacto vía WhatsApp.

2.2. Exclusiones Explícitas (Fuera de Alcance)

Para mitigar riesgos de desarrollo y asegurar la entrega en los tiempos estipulados, las siguientes características **no** forman parte de la solución actual:

- Procesamiento de pagos en línea (Pasarelas de pago externas).
- Chat de mensajería interna entre usuarios y administración.
- Mapas interactivos con geolocalización en tiempo real.

3. Arquitectura Técnica y Stack Tecnológico

La solución adopta una arquitectura desacoplada basada en el patrón de diseño **Client-Server**, estructurada en un monorepositorio para facilitar el despliegue continuo.

```
[ Frontend: React 19 ] —( Peticiones HTTP / JSON )—> [ Backend: Spring Boot 3.5 ] —> [ Base de Datos: MariaDB ]
```

3.1. Infraestructura Tecnológica

Capa	Tecnología
Backend	Java 17 / Spring Boot 3.5 / Spring Data JPA / Lombok
Frontend	React 19 / Vite / React Router (Single Page Application - SPA)
Base de Datos	MariaDB (Motor Relacional)
Seguridad (Sprint 2+)	Spring Security + JSON Web Tokens (JWT)
Protocolo	API RESTful en <code>http://localhost:8080/api/</code>

3.2. Estándar de Diseño del Frontend

La interfaz se construye con **CSS Puro** mediante **Variables Dinámicas (Custom Properties)**. Las variables están preparadas para soportar un intercambio de temas claro/oscuro mediante el atributo `data-theme` en el DOM. La tabla siguiente muestra los valores definidos para cada tema:

Elemento	Paleta Light (Clara)	Paleta Dark (Oscura)
Background (Fondo)	#f4f4f9	#05050a
Text (Texto Principal)	#333333	#cccccc
Primary (Acciones/Botones)	#ff6b35	#ff5500
Secondary (Componentes/Nav)	#264653	#abccd8
Accent (Destacados/Badges)	#2a9d8f	#62d5c8

Referencia visual de la paleta: [Paleta de colores TuHospedaje](#).

4. Decisiones Arquitectónicas Clave (ADR)

4.1. Uso del Dominio Semántico `Lodging`

En lugar de utilizar términos genéricos como `Product`, todo el backend y frontend se alinean bajo el concepto de **Lodging** (Alojamiento). Esto asegura la coherencia del código con el modelo de negocio.

4.2. Modelo de Datos Extensible desde el Día 1

La propiedad `Lodging.category` se diseña como una Foreign Key de tipo **nullable** en el Sprint 1. Esto permite poblar la base de datos de manera inmediata sin requerir migraciones complejas de esquemas (como scripts Flyway o Liquibase) al implementar el Sprint 2.

4.3. Patrón de Backend Desacoplado

Se implementa un flujo estricto:

```
Controller → Service (Interface + Impl) → Repository → Entity / DTO
```

Los controladores se inyectan estrictamente a través de constructores (evitando la directiva `@Autowired` en atributos para facilitar pruebas unitarias), y la conversión de datos se centraliza en los DTOs mediante métodos estáticos de mapeo (`toEntity()` / `fromEntity()`).

4.4. Inyección de Configuración mediante Entornos

El Frontend se desacopla del entorno de ejecución utilizando variables de entorno (`.env`) mediante el prefijo `VITE_API_URL` , asegurando que la transición a servidores de producción sea transparente.